

Laboratory Work No. 3

Training Object Detectors

Student Name: [Your Name]
Group/ID: [Your Group]
Variant No.: [e.g., 1]
Dataset: [e.g., People Detection]
Detector: [e.g., YOLO11n]
Instructor: Assoc. Prof. A. Kroshchanka
Date: September 8, 2026

Brest State Technical University
Department of Intellectual Information Technologies

Abstract

This report details the process of training a neural network object detector using the YOLO (You Only Look Once) architecture. The objective is to configure a custom dataset, organize the training pipeline for a specific YOLO variant, and evaluate the model's performance using standard metrics such as mean Average Precision (mAP). Finally, the trained detector is tested on arbitrary images sourced from the Internet to visualize its real-world inference capabilities.

Contents

1	Introduction and Objectives	2
2	Variant Details	2
3	Data Preparation	2
4	Model Training	3
5	Evaluation Results	3
6	Inference Visualization	3
7	Conclusion	4
A	Assignment Variants	5

1 Introduction and Objectives

Laboratory Goal

To perform the training of a neural network detector for solving the task of detecting specified objects.

The specific tasks to be completed in this laboratory work are:

1. Explore the assigned dataset, performing any necessary data transformations to organize the training process.
2. Organize the training process for the assigned neural network detector architecture on the specified dataset.
3. Evaluate the training efficiency on the test set using the mean Average Precision (mAP) metric.
4. Implement visualization of the detector's operation by detecting objects on individual photographs retrieved from the Internet.

2 Variant Details

According to the assignment, the experiment is conducted using the configuration presented in Table 1.

Table 1: Assigned Configuration

Parameter	Value
Variant Number	[Insert Variant, e.g., 1]
Dataset Topic	[e.g., People]
Dataset Source	[InsertRoboflowURL]
Detector Architecture	[e.g., YOLO11n]

3 Data Preparation

The dataset was obtained from Roboflow and exported in the YOLO format. The dataset structure includes `images` and `labels` directories for training, validation, and testing splits, along with a `data.yaml` file that defines the number of classes and their corresponding paths.

```

1 # Example of data.yaml structure
2 train: ../train/images
3 val: ../valid/images
4 test: ../test/images
5
6 nc: 1 # number of classes
7 names: ['person'] # class names

```

Listing 1: Dataset Configuration (data.yaml example)

4 Model Training

The training process was organized using the `ultralytics` Python package. The assigned YOLO model architecture was initialized with pre-trained weights to speed up convergence (transfer learning).

```

1 from ultralytics import YOLO
2
3 # Load the assigned YOLO model (e.g., YOLO11n)
4 model = YOLO('yolo11n.pt')
5
6 # Train the model on the custom dataset
7 results = model.train(
8     data='path/to/data.yaml',
9     epochs=50,
10    imgsz=640,
11    batch=16,
12    name='custom_detector'
13 )

```

Listing 2: YOLO Training Pipeline

5 Evaluation Results

After training, the model's performance was evaluated on the test set. The primary metric for object detection is the mean Average Precision (mAP).

Table 2: Evaluation Metrics on Test Set

Class	Precision	Recall	mAP@50
All	0.XX	0.XX	0.XX

The obtained mAP score indicates that the model successfully learned to identify and localize the target objects with a high degree of accuracy.

6 Inference Visualization

To verify the detector's practical application, arbitrary photographs related to the dataset topic were retrieved from the Internet. The trained model was used to predict bounding boxes and class probabilities.

```

1 # Load the fine-tuned custom model
2 best_model = YOLO('runs/detect/custom_detector/weights/best.pt')
3
4 # Perform inference on an image from the Internet
5 results = best_model.predict('https://example.com/test_image.jpg', save=
    True, conf=0.5)

```

Listing 3: Model Inference

As seen in Figure X, the detector successfully placed bounding boxes around the target objects, correctly applying the learned features to unseen data.

7 Conclusion

In this laboratory work, an object detector based on the YOLO architecture was successfully trained. The pipeline included data acquisition and formatting, model fine-tuning, and evaluation using standard object detection metrics (mAP). The final trained model demonstrated robust capability in identifying and localizing specific objects in real-world images.

References

- [1] Ultralytics. (2024). *YOLO Docs*. Retrieved from <https://docs.ultralytics.com/models/>
- [2] Roboflow Universe. (2024). *Computer Vision Datasets*. Retrieved from <https://universe.roboflow.com/>

A Assignment Variants

Table 3 lists all configurations for this laboratory work.

People:

<https://universe.roboflow.com/leo-ueno/people-detection-o4rdr/dataset/10>

License Plates:

<https://universe.roboflow.com/roboflow-universe-projects/license-plate-recognition-rx/dataset/11>

Vehicles:

<https://universe.roboflow.com/roboflow-100/vehicles-q0x2v/dataset/2>

Cats:

<https://universe.roboflow.com/mohamed-traore-2ekkp/cats-n9b87/dataset/3>

Water Meters:

<https://universe.roboflow.com/koer3741-gmail-com/watermeteramrv2/dataset/1>

Rock-Paper-Scissors:

<https://universe.roboflow.com/roboflow-58fyf/rock-paper-scissors-sxsw>

Table 3: Variants for Laboratory Work 3

Variant	Detector	Dataset (Roboflow)
1	YOLO11n	People
2	YOLO11n	License Plates
3	YOLO11s	Vehicles
4	YOLO11s	Cats
5	YOLO11m	Water Meters
6	YOLO11m	Rock-Paper-Scissors
7	YOLO12n	People
8	YOLO12n	License Plates
9	YOLO12s	Vehicles
10	YOLO12s	Cats
11	YOLO12m	Water Meters
12	YOLO12m	Rock-Paper-Scissors
13	YOLO26n	People
14	YOLO26n	License Plates
15	YOLO26s	Vehicles
16	YOLO26s	Cats
17	YOLO26m	Water Meters
18	YOLO26m	Rock-Paper-Scissors